Propuesta de MVP: Evaluación de competencias del perfil de egreso a partir de informes de práctica

Grupo: 2

Integrantes:

Sofia Castelli

Carolina Hong

Clara Leighton

Profesora:

Carla Vairetti

5 de Mayo de 2026

1. Problema a Resolver

La Universidad de los Andes enfrenta desafíos en la gestión curricular, especialmente en la capacidad de verificar, de manera sistemática, trazable y escalable, el cumplimiento del perfil de egreso de sus estudiantes. En el caso de las prácticas profesionales y pre profesionales, la evaluación actual opera principalmente como un proceso binario: el estudiante aprueba o reprueba la práctica en función de la revisión de su informe.

Sin embargo, esta aprobación no permite conocer con claridad qué competencias del perfil de egreso fueron efectivamente evidenciadas ni en qué nivel. Esto genera una brecha importante para el área curricular, ya que limita la posibilidad de hacer seguimiento al desarrollo de habilidades, detectar vacíos formativos y tomar decisiones basadas en evidencia.

Los principales problemas son:

- Aprobación como criterio único: actualmente, aprobar el informe se interpreta como evidencia suficiente del cumplimiento del perfil de egreso, aunque no exista un desglose por competencia.
- Alta subjetividad: la evaluación depende del criterio de cada revisor, por lo que dos evaluadores podrían interpretar de forma distinta el mismo informe.
- Sin trazabilidad longitudinal: los resultados no quedan estructurados por competencia, lo que impide observar cómo evoluciona un estudiante desde la pre práctica, práctica profesional, memoria o proyecto de título.
- Baja escalabilidad: revisar manualmente todos los informes de una generación completa requiere mucho tiempo, especialmente si se busca identificar evidencia específica para cada competencia.

Por lo tanto, el problema central no es sólo corregir informes, sino transformar esos documentos en evidencia estructurada que permita monitorear el cumplimiento del perfil de egreso.

2. Solución Propuesta

Se propone un MVP que funcione como una herramienta de apoyo para el área curricular. El sistema recibe informes de práctica en formato digital, identifica evidencia dentro del texto y la relaciona con las competencias del perfil de egreso definidas en una matriz de competencias. El MVP no reemplaza al evaluador humano. Su objetivo es generar una primera evaluación estructurada, editable y trazable, que permita al usuario revisar, corregir y validar los resultados.

La solución se basa en un pipeline de análisis documental compuesto por siete capas, desde la carga del documento hasta la generación de una vista preliminar de resultados. Una vez revisada y validada por el usuario, esta información se exporta como un archivo final en formato .xlsx, compuesto por una hoja de resultados por competencia y una hoja de reporte de procesamiento.

2.1 Alcance del MVP

Para mantener la solución acotada y abordable, el MVP se plantea como una aplicación local e interactiva desarrollada en Python, utilizando herramientas como Streamlit para construir una interfaz simple de carga, revisión y validación. Esta decisión permite implementar y probar el flujo principal del sistema sin requerir el desarrollo de una plataforma web permanente o infraestructura de despliegue compleja. Sin embargo, el diseño sigue considerando que su público objetivo requiere una interfaz amigable y fácil de usar, que permita interactuar con los resultados sin necesidad de conocimientos técnicos avanzados.

El MVP se enfoca en procesar informes individuales de práctica mediante una aplicación local e interactiva. Su alcance se limita a ejecutar el flujo completo para un informe a la vez: cargar el documento, analizarlo mediante el pipeline propuesto, presentar una vista preliminar revisable y generar una exportación final validada.

Esta herramienta busca incorporar al proceso de revisión una capa adicional de análisis que permita observar el cumplimiento del perfil de egreso con mayor precisión. De esta forma, el MVP habilita una evaluación granular que actualmente no se realiza de manera sistemática, entregando evidencia, niveles preliminares, justificaciones y trazabilidad para apoyar una revisión más robusta. No busca reemplazar al evaluador, sino más bien agregar un análisis adicional a su revisión.

En particular, el MVP incluye las siguientes funcionalidades:

- Carga de informes de práctica en formato PDF.
- Uso de una matriz de competencias en formato CSV y un archivo JSON estructural del informe.
- Extracción de texto del informe.
- Identificación de secciones, segmentación en chunks y recuperación de evidencia.
- Evaluación por competencia mediante un pipeline de siete capas.
- Generación de una vista preliminar editable y revisable por el evaluador.
- Incorporación de un mecanismo Human-in-the-Loop para aceptar, modificar o rechazar resultados.
- Registro de trazabilidad del procesamiento y datos necesarios para el cálculo posterior de KPI.
- Exportación final en formato .xlsx, con resultados por competencia y reporte de procesamiento.

Esta versión inicial del MVP se concentra exclusivamente en la validación del pipeline de evaluación, por lo que ciertos componentes quedan deliberadamente fuera de su alcance. En el ámbito técnico y de interfaz, no contempla un despliegue web permanente ni el desarrollo de entornos gráficos complejos, priorizando la estabilidad del procesamiento automatizado. De igual forma, se excluye el entrenamiento o ajuste fino de modelos propios, ya que la arquitectura opera con sistemas pre entrenados y técnicas de recuperación semántica listas para uso. Respecto al tratamiento de documentos, el análisis se limita estrictamente a texto plano extraído digitalmente, dejando fuera la interpretación de contenido no estructurado como imágenes, anexos escaneados o tablas de estructura compleja.

Por último, la mayor proyección a futuro, que no entra en el alcance del MVP inicial, es la extensión a otras especialidades de la carrera y el análisis agregado del cumplimiento macro del perfil de egreso en Ingeniería Civil en Ciencias de la Computación. Estos elementos podrían incorporarse en etapas posteriores, una vez validado el funcionamiento del pipeline principal y la utilidad del MVP para el área curricular.

3. Flujo del sistema

El sistema opera bajo un modelo de activación pasiva donde el flujo inicial se desencadena automáticamente al cargar únicamente el informe del estudiante. Los archivos de configuración, como la matriz de competencias y la rúbrica, se gestionan de manera persistente en el sistema; una vez cargados inicialmente, permanecen disponibles para futuras evaluaciones. A partir de este punto, se ejecuta el pipeline de 7 capas de manera secuencial hasta generar una vista preliminar de resultados. Solo después de que esta vista es revisada y validada por el usuario, se genera la salida final en formato .xlsx.

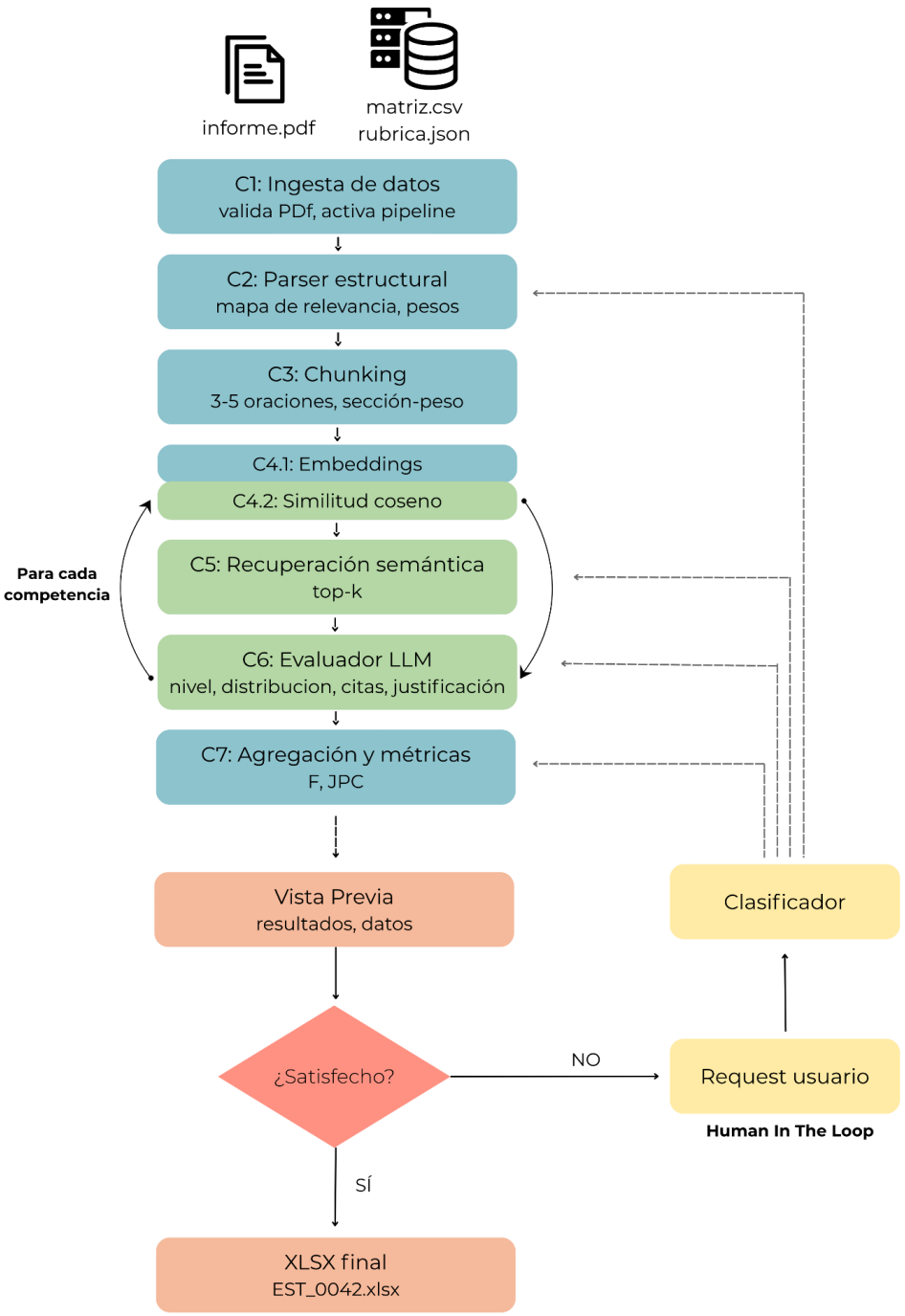

Figura 1: Arquitectura del pipeline de evaluación
Fuente: Elaboración propia

La arquitectura se distingue por integrar un mecanismo de Human-in-the-Loop (HITL) inspirado en las conexiones residuales (ResNet). Este diseño permite que las instrucciones del usuario "salten" (skip connections) directamente a la capa específica donde se originó el dato, retomando el cálculo en cascada desde ese punto para garantizar la consistencia de los resultados.

Una vez obtenida la vista preliminar, el usuario revisa la evaluación propuesta por el sistema. Si algún resultado no le parece suficientemente claro, coherente o respaldado, puede solicitar una revisión o ajuste desde la misma interfaz. Por ejemplo, puede pedir modificar el nivel de una competencia, agregar evidencia, corregir una sección

mal detectada o solicitar una nueva revisión. Estas solicitudes no se aplican manualmente sobre el archivo final, sino que son interpretadas por el sistema y redirigidas a la etapa del pipeline donde corresponde realizar el cambio.

A partir de ese ajuste, el sistema actualiza la vista preliminar y vuelve a calcular los elementos que dependan de la modificación realizada. De esta manera, se mantiene la consistencia entre la evidencia utilizada, el nivel asignado, la justificación, la confianza y el resultado consolidado, sin necesidad de reprocesar todo el flujo desde cero.

Durante este ciclo de revisión, el sistema no genera un nuevo archivo Excel por cada iteración. Los ajustes actualizan el estado interno de la aplicación y se reflejan directamente en la vista preliminar. El usuario puede repetir este proceso hasta considerar que la evaluación es correcta. Solo cuando el usuario valida el resultado, el sistema genera la salida final en formato .xlsx, compuesta por dos hojas: una con los resultados por competencia y otra con el reporte de procesamiento del informe. De esta manera, el archivo Excel funciona como una exportación final validada, y no como un archivo intermedio que se genera en cada iteración.

De esta manera, el MVP funciona bajo una lógica de validación humana, donde el sistema automatiza el procesamiento inicial del informe, pero el evaluador mantiene el control final sobre la salida. En las siguientes secciones se describe en detalle el funcionamiento de cada capa del pipeline y el rol que cumple dentro de este flujo general.

Loop iterativo por competencia

El procesamiento entre las capas 4, 5 y 6 combina una etapa global y una etapa iterativa por competencia a evaluar.

Fase inicial: Generación de embeddings
Una vez que la Capa 3 ha completado la segmentación del informe en chunks, la Capa 4 genera los embeddings iniciales (representaciones vectoriales) de todos los chunks del informe una única vez, así como los embeddings de todas las competencias de la matriz. Estos embeddings se almacenan como valores de referencia constantes que permanecen sin cambios durante todo el procesamiento. Esta fase inicial no es iterativa; se ejecuta una sola vez al inicio.

Fase iterativa: loop por competencia
A continuación, el sistema inicia el loop iterativo. Para cada competencia en la matriz reducida, se ejecuta un ciclo que atraviesa las capas 4, 5 y 6 de manera secuencial.

En la primera iteración del loop para una competencia específica, la Capa 4 calcula la similitud coseno entre el embedding de esa competencia y los embeddings de todos los chunks del informe. Este cálculo genera un conjunto de valores numéricos que cuantifican el grado de alineación semántica entre esa competencia y cada fragmento del documento.

Los valores de similitud coseno calculados en la Capa 4 se propagan hacia las capas posteriores, donde son utilizados para guiar el procesamiento específico de esa competencia. La Capa 5 utiliza estos valores de similitud para recuperar y priorizar la evidencia más relevante. La Capa 6 recibe esta evidencia recuperada, ordenada por similitud, para realizar la evaluación final de la competencia.

Una vez que la Capa 6 completa la evaluación de una competencia, el sistema almacena el resultado y avanza a la siguiente competencia en la matriz reducida. El loop se repite iterativamente, para cada competencia, hasta que todas las competencias han sido procesadas.

Salida del loop

Al finalizar el loop iterativo, el sistema ha generado una evaluación completa para cada competencia de la matriz. Todos estos resultados se consolidan y se preparan para la Capa 7 (Agregación), donde se organizan, se calculan las métricas finales de confianza y sustento, y se preparan para la revisión del usuario

En las siguientes secciones se describe en detalle el funcionamiento de cada capa del pipeline y el rol que cumple dentro de este flujo general.

3.1 Descripción del funcionamiento por capas

El funcionamiento del MVP se estructura como un pipeline de procesamiento dividido en siete capas. Cada capa cumple una función específica dentro del flujo, permitiendo transformar un informe de práctica en un resultado estructurado por competencias.

3.1.1 Capa 1: Ingesta de datos

En esta primera capa, el sistema recibe los datos de entrada necesarios para ejecutar el análisis. Recibe el informe del estudiante en formato PDF (con sus metadatos asociados correspondientes), junto con los archivos de configuración disponibles en el sistema: la matriz de competencias en formato CSV y un archivo JSON que describe la estructura del informe en detalle. En el uso normal de la herramienta, el usuario carga únicamente el informe a evaluar, mientras que la matriz y la rúbrica permanecen almacenadas de manera persistente como configuración del sistema.

La matriz de competencias define qué competencias del perfil de egreso deben ser evaluadas según la carrera y el tipo de práctica del estudiante. En particular, distingue entre práctica pre-profesional y práctica profesional, marcando con una “x” las competencias que corresponden a cada caso, como se ve ejemplificado en la Figura 2 a continuación. Por esto, antes de iniciar la evaluación, el sistema identifica el tipo de práctica asociado al informe a partir de los metadatos de este. Con esa información, filtra la matriz original y construye una matriz reducida que contiene únicamente las competencias que deben ser evaluadas para ese tipo de práctica.

	UNIVERSIDAD				LICENCIATURA INGENIERÍA CIVIL					LTURA INGENIERÍA CIVIL COMP		TITULACIÓN INGENIERÍA CIVIL					TITULACIÓN INGENIERÍA CIVIL COMPUTACION		
	Con destrezas básicas consolidadas: pensar, hablar, leer críticamente, escribir e interactuar, trabajar en equipo	Con fuerte deseo de servicio a la sociedad	Con conocimientos básicos de los fundamentos de la fe católica y su coherencia intrínseca con la razón	Capaces de discernir con sentido crítico los paradigmas de la sociedad en la que viven y trabajan	Utiliza conocimientos y metodologías de ciencias básicas y ciencias de la ingeniería, formulando problemas de ingeniería que combinan la teoría y la práctica en su solución.	Aplica las técnicas y métodos de las ciencias de la ingeniería a la resolución de problemas fundamentales, utilizando nuevas tecnologías y tomando en cuenta los supuestos y limitaciones teóricas y prácticas.	Integra eficazmente equipos de trabajo, estableciendo metas, planificando tareas y cumpliendo objetivos en los plazos requeridos.	Se comunica por escrito y oralmente de forma efectiva tanto en español como en inglés, especialmente en temas relativos a su profesión.	Reconoce responsabilidades éticas y profesionales en problemas de ingeniería, considerando el impacto de las soluciones en contextos técnicos, económicos, ambientales y sociales.	Demuestra conocimientos de ciencias de la computación y la capacidad para aplicarlos en la solución de problemas susceptibles de ser resueltos computacionalmente.	Desarrolla software para la solución de problemas de distinta índole, utilizando varios lenguajes y plataformas en el estado del arte.	Diseñar un sistema, componente o proceso para satisfacer una necesidad cumpliendo requisitos definidos y restricciones de ámbito diverso, como económicas, ambientales, sociales, éticas, de seguridad, entre otras.	Identifica y resuelve problemas complejos de ingeniería civil, de forma sistemática y/o creativa, aplicando principios de ciencias de la ingeniería, ciencias básicas y matemáticas, con ausencia de datos completos.	Posee las habilidades para tomar decisiones fundamentadas en ámbitos correspondientes a su quehacer profesional.	Lidera equipos de trabajo multidisciplinarios asegurando el cumplimiento de los objetivos.	Reconoce los problemas y responsabilidades asociados a la práctica de la ingeniería, comprometiéndose con la ética profesional y las responsabilidades que rigen su profesión.	Lidera el proceso de análisis necesario para el desarrollo de sistemas de software, enfocándose en las necesidades humanas, organizacionales, de negocio, y tecnológicas de los usuarios e interesados.	Diseña software, incluyendo su arquitectura, diseño detallado e incorpora buenas prácticas al diseño.	Negocia, planifica y gestiona un proyecto de desarrollo de software en toda su extensión, con preocupación por la calidad del software y la entrega de valor a los usuarios e interesados.
Pre-Professional Practice	x	x	x	x	x	x	x	x	x	x	x								
Professional Practice	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x

Figura 2: Extracto de Matriz Tributación
Fuente: Elaboración propia

La rúbrica, por su parte, establece la estructura esperada del informe y el peso relativo de cada sección dentro de la evaluación. En el MVP, esta información no sería inferida automáticamente por el sistema, sino que se recibiría como un archivo JSON previamente construido. Este JSON se elaboraría a partir de dos fuentes: la ***plantilla oficial del informe de práctica***, que define las secciones esperadas y sus contenidos, y el ***resumen de ponderaciones***, que indica el peso asignado a cada criterio evaluado.

De esta forma, el archivo JSON funciona como una configuración estructural del sistema: contiene los nombres de las secciones, sus subsecciones, títulos esperados y las ponderaciones asociadas a cada criterio. Esto permite que el

sistema identifique qué partes del informe debe buscar y cuánto peso relativo tiene cada una, sin dejar esta información fija dentro del código. Un ejemplo simplificado de esta estructura sería:

```
{
  "pre_professional_practice": {
    "abstract": {
      "peso": 0.10,
      "subsecciones": {
        "abstract": 0.10
      }
    },
    "introduccion": {
      "peso": 0.16,
      "subsecciones": {
        "department_area_description": 0.05,
        "company_description":         0.05
      }
    },
    ...
  },}
```

Bloque de código 1: Representación conceptual Rúbrica JSON
Fuente: Elaboración propia

En esta etapa también se realiza la extracción de texto del documento. Este paso es crítico, ya que todo el procesamiento posterior depende de contar con texto digital.

Como resultado, esta capa tiene como output un objeto estructurado con la información ya preparada para continuar el procesamiento. Este objeto incluye el texto extraído del PDF, los metadatos asociados al informe, la lista reducida de competencias que corresponde evaluar, incluyendo su identificador, nombre y descripción, y la configuración activa del informe obtenida desde el archivo JSON.

Además, en esta etapa se inicializa el reporte de procesamiento del informe. Este reporte no corresponde a la evaluación del estudiante, sino a una estructura de trazabilidad que se irá completando durante las siguientes capas del pipeline. En esta primera capa se registran los metadatos del informe, el tipo de práctica detectado y la lista reducida de competencias que deberán ser evaluadas.

3.1.2 Capa 2: Parser estructural

Una vez obtenido el texto extraído en la Capa 1, el sistema organiza el contenido del informe identificando sus distintas secciones. Para ello, utiliza la configuración activa del informe generada a partir del archivo JSON.

```
{
  "secciones_informe": {
    "abstract": {
       "texto": "TechCorp S.A. es una empresa...",
       "peso": 0.1
    }
    "2.1": {
       "texto": "Durante las 12 semanas...",
       "peso": 0.2
    }
    "3.1": {...}
    "3.2": {...}
    "4": {...}
  },
 "secciones_ausentes" : {
    "2.2": {"peso": 0.2}
  }
}
```

Bloque de código 2: Representación conceptual de división del informe por sección
Fuente: Elaboración propia

Cada bloque de texto es asignado a su sección correspondiente, como por ejemplo *Abstract*, *General Activities*, *Most Challenging Activit*y o *Personal Reflection*. Esta estructuración permite transformar el documento en una representación más ordenada y comprensible para el sistema.

En caso de que alguna sección esperada no sea detectada, el sistema continúa el proceso utilizando únicamente las secciones disponibles, pero registra qué sección no fue encontrada. En esta etapa se generan dos estructuras internas: la primera contiene el **texto del informe dividido por secciones**, mientras que la segunda corresponde a la **actualización del reporte de procesamiento** con la información del parser. En esta actualización se enumeran las secciones detectadas y las secciones ausentes. Esta información no forma parte del contenido evaluado, pero permite mantener trazabilidad sobre la estructura del documento y revisar posteriormente si la ausencia de alguna sección pudo afectar el sustento de la evaluación.

```
{
  "secciones_informe": [
    "abstract",
    "2.1",
    "3.1",
    "3.2",
    "4"
  ],
  "secciones_ausentes": [
    "2.2"
  ],
}
```

Bloque de código 3: Representación conceptual de actualización del reporte sobre secciones
Fuente: Elaboración propia

Adicionalmente, en esta capa se construye un **mapa de relevancia** que indica qué secciones son más importantes para evaluar cada competencia. Este mapa se genera considerando las competencias activas entregadas por la Capa 1, la estructura esperada del informe y las secciones efectivamente disponibles después del parser. Esto permite diferenciar entre secciones principales, secundarias y contextuales para cada habilidad, de modo que el sistema priorice aquellas partes del informe con mayor probabilidad de contener evidencia sustantiva y utilice las secciones menos relevantes solo como apoyo contextual.

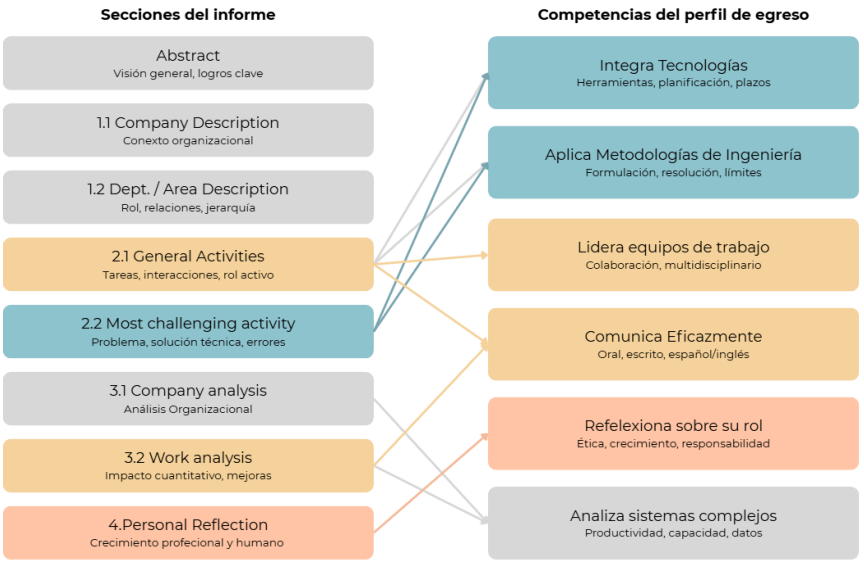

Figura 3: Visualización de mapa de relevancia
Fuente: Elaboración propia

Por tanto, como output, la Capa 2 entrega tres estructuras internas: el **texto del informe dividido por secciones**, el **mapa de relevancia** y la **actualización del reporte de procesamiento** con los resultados del parser.

3.1.3 Capa 3: Segmentación del texto (Chunking)

En esta capa, el sistema recibe como entrada el texto del informe ya organizado por secciones en la Capa 2. A partir de esta estructura, cada sección es dividida en fragmentos más pequeños de texto, generalmente de tres a cinco oraciones. Esta segmentación permite transformar secciones extensas en unidades de análisis más manejables para las etapas posteriores.

Cada fragmento conserva información clave sobre su origen dentro del documento, incluyendo la sección a la que pertenece, el peso asociado a dicha sección y su posición dentro del informe. Esta información es fundamental, ya que permite mantener trazabilidad sobre el origen de cada fragmento y utilizar su contexto en las etapas posteriores del análisis.

A continuación se muestra un ejemplo de cómo se divide la sección 2.2 del informe en chunks. Cabe destacar que los pesos se heredan desde la estructura de secciones generada como output de la Capa 2, donde cada sección ya contiene su texto y peso correspondiente [Bloque de código 2]

```
c01 · sec 2.2 · peso 0.35
"La actividad más desafiante fue diseñar una API REST para trazabilidad
de envíos. El sistema anterior consultaba la BD directamente desde el
frontend, generando cuellos de botella..."

c02 · sec 2.2 · peso 0.35
"Implementé los endpoints con FastAPI, definiendo contratos con OpenAPI.
Añadí autenticación JWT, rate limiting y manejo de errores HTTP
estandarizado (4xx/5xx)..."

c03 · sec 2.2 · peso 0.35
"El mayor desafío técnico fue el diseño del esquema de caché.
Evalué Redis vs Memcached, considerando consistencia eventual
vs fuerte dado que los datos de envío se actualizan frecuentemente..."
```

Bloque de código 4: Representación conceptual de división de una sección en chunks
Fuente: Elaboración propia

Como output, la Capa 3 entrega una lista de chunks etiquetados. Cada elemento de esta lista contiene el texto del fragmento, su identificador, la sección de origen, el peso de la sección y su posición dentro del documento. Esta lista será utilizada por la Capa 4 para generar los embeddings de cada chunk. Además, la información de sección y peso se mantiene asociada a cada fragmento, ya que será necesaria posteriormente para priorizar la recuperación de evidencia y mantener trazabilidad en la salida final.

3.1.4 Capa 4: Representación semántica (embeddings)

Esta capa se encarga de transformar el texto procesado en representaciones matemáticas vectoriales, permitiendo al sistema comparar el contenido semántico de manera numérica. Recibe como entrada la lista de chunks etiquetados generada en la Capa 3, junto con la lista reducida de competencias a evaluar, donde cada competencia contiene su identificador, nombre y descripción textual.

Fase 1: Generación inicial de embeddings
Internamente, esta capa puede entenderse en dos partes. Primero, se generan los embeddings de todos los chunks del informe. Este proceso se realiza una sola vez, ya que los mismos fragmentos serán reutilizados para evaluar todas las competencias. Segundo, se generan los embeddings de las competencias activas, utilizando sus descripciones textuales. Esto permite representar en el mismo espacio vectorial tanto la evidencia potencial del informe como las competencias que se buscan evaluar.

```
vec_competencia_C13 = embed(
    "Integra eficazmente tecnologías y herramientas de trabajo,
     estableciendo metas, planificando tareas y cumpliendo
     objetivos en los plazos requeridos.")
  → [0.021, -0.143, 0.087, 0.312, ...] (1536 dims)

  vec_c04 = embed("La actividad más desafiante fue diseñar una API REST...")
  vec_c05 = embed("Implementé endpoints con FastAPI, OpenAPI, JWT...")
  vec_c06 = embed("Evalué Redis vs Memcached considerando consistencia...")
  vec_c09 = embed("La API redujo tiempos en 40%. Error rate 12% → 0.3%...")
  vec_c12 = embed("Aprendí que el trade-off entre simplicidad y robustez...")
  vec_c01 = embed("TechCorp S.A. es una empresa de logística con 300 empleados…”)
```

Bloque de código 5: Representación conceptual de transformación de chunks y competencias a embeddings
Fuente: Elaboración propia

El proceso proyecta cada chunk y cada descriptor de competencia en un espacio vectorial de alta dimensionalidad. En este espacio, la proximidad entre dos vectores indica un mayor grado de similitud semántica, independientemente de las palabras exactas utilizadas. De esta manera, conceptos como “diseño de API” y “creación de servicios web” pueden quedar representados por vectores cercanos, facilitando la recuperación posterior de evidencia relevante.

Todos estos embeddings se almacenan como valores de referencia constantes que permanecen sin cambios durante todo el procesamiento.

Fase 2: Loop iterativo - Cálculo de Similitud Coseno por competencia

Una vez generados y almacenados los embeddings iniciales, el sistema inicia un loop iterativo que se ejecuta para cada competencia de la matriz. En cada iteración:

Para la competencia siendo evaluada en esa iteración, la Capa 4 calcula la similitud coseno entre el embedding de esa competencia y los embeddings de todos los chunks del informe. Este cálculo genera un conjunto de valores

numéricos que indican qué tan alineado semánticamente está cada fragmento del documento con la competencia específica siendo evaluada.

Estos valores de similitud coseno son específicos para cada competencia y serán utilizados por la Capa 5 para ordenar y filtrar los fragmentos más relevantes. Este proceso permite comparar textos en función de su contenido semántico en lugar de depender de coincidencias exactas de palabras, asegurando que la evaluación no dependa de términos específicos, sino del significado del contenido del informe.

Una vez que la Capa 5 y Capa 6 completan el procesamiento de esa competencia, el loop avanza a la siguiente competencia en la matriz reducida, donde se calculan los valores de similitud coseno para esa nueva competencia. Este proceso se repite iterativamente hasta que todas las competencias han sido evaluadas.

Como output, la Capa 4 entrega una estructura semántica con dos niveles. Primero, mantiene una representación global de los chunks del informe, donde cada fragmento queda asociado a su embedding y a su metadata original. Segundo, genera una estructura por competencia, donde cada competencia queda asociada a su embedding y a los puntajes de similitud coseno calculados contra todos los chunks del informe. Esta segunda estructura es la que se entrega a la Capa 5 en cada iteración del procesamiento por competencia, permitiendo que la recuperación de evidencia se realice de manera específica para la competencia evaluada.

Además, esta capa actualiza el reporte de procesamiento con información de trazabilidad por competencia. En particular, registra si el embedding de cada competencia fue generado correctamente, si se calcularon sus similitudes coseno contra los chunks del informe y cuántos fragmentos fueron considerados en esa comparación. Esta información no corresponde a la evaluación del estudiante ni incluye los vectores completos, sino que permite verificar que cada competencia de la matriz reducida fue efectivamente incorporada al proceso semántico. Estos datos serán utilizados posteriormente para calcular la cobertura del pipeline y distinguir entre competencias procesadas correctamente y competencias omitidas por error.

```
{ "trazabilidad_competencias": [
    {
      "competencia_id": "C13",
      "embedding_generado": true,
      "similitud_calculada": true,
      "chunks_comparados": 31,
      "estado_capa_4": "OK"
    },
    {
      "competencia_id": "C9",
      "embedding_generado": true,
      "similitud_calculada": true,
      "chunks_comparados": 31,
      "estado_capa_4": "OK"
    } ]
}
```

Bloque de código 6: Reporte de procesamiento de información de la Capa 4
Fuente: Elaboración propia

3.1.5 Capa 5: Recuperación de evidencia

En esta etapa, el sistema identifica los fragmentos del informe más relevantes para evaluar cada competencia. Para ello, recibe como input la estructura generada en la Capa 4, que contiene los puntajes de similitud coseno entre la competencia evaluada y cada chunk del informe. Además, utiliza el mapa de relevancia construido en la Capa 2, [Figura 3] donde se indica qué secciones funcionan como fuentes principales, secundarias o contextuales para cada competencia.

La Capa 5 opera dentro del loop por competencia. Esto significa que, en cada iteración, recibe la información correspondiente a una competencia específica: su identificador, nombre, descripción, los puntajes de similitud con los chunks del informe y el mapa de relevancia asociado a esa competencia. A partir de esa información, la capa recupera un conjunto acotado de evidencia candidata para ser enviado a la Capa 6.

El proceso no busca evidencia en todo el documento con la misma prioridad. Para cada competencia, el sistema comienza revisando los chunks que pertenecen a las secciones definidas como fuentes principales en el mapa de relevancia. Dentro de estas secciones, utiliza los puntajes de similitud coseno calculados en la Capa 4 para ordenar los fragmentos desde los más pertinentes hasta los menos pertinentes.

Cabe destacar que un fragmento se considera pertinente si su similitud coseno con la competencia es mayor o igual a 0.65. Sin embargo, encontrar un único fragmento sobre este umbral no se considera evidencia suficiente. Para evitar que la evaluación dependa de una sola mención aislada, el sistema busca reunir una cantidad mínima de fragmentos pertinentes en las fuentes principales. Además, se define un máximo top_k de fragmentos por competencia, de modo que, si existen muchos chunks relevantes, sólo se seleccionan aquellos con mayor similitud para evitar sobrecargar al evaluador LLM.

Si el sistema logra reunir esta cantidad mínima de fragmentos pertinentes en las fuentes principales, selecciona los mejores chunks y no expande la búsqueda a otras secciones. Si no logra reunir suficiente evidencia sobre el umbral, amplía la búsqueda a fuentes secundarias y, en última instancia, a secciones contextuales.

Este procedimiento permite evitar que fragmentos genéricos, como los del *Abstract* o *Company Description*, desplacen evidencia más sustantiva proveniente de secciones como *Most Challenging Activity* o *Work Analysis*. Así, la recuperación no depende únicamente de qué fragmento es semánticamente parecido a la competencia, sino también de si proviene de una sección donde se espera encontrar evidencia relevante.

Como output, la Capa 5 entrega un conjunto reducido de fragmentos recuperados para la competencia evaluada en esa iteración. Cada fragmento conserva su texto, identificador, sección de origen, tipo de fuente, peso asociado y puntaje de similitud coseno. Esta salida corresponde a la evidencia candidata que será enviada a la Capa 6 para que el LLM evalúe esa competencia.

```
{
  "competencia_id": "C13",
  "competencia_nombre": "Desarrolla software",
  "evidencia_recuperada": [
    {
      "chunk_id": "c05",
      "texto": "Implementé endpoints con FastAPI, OpenAPI, JWT...",
      "seccion": "2.2",
      "tipo_fuente": "principal",
      "peso_fuente": 1.0,
      "similitud": 0.89
    },
```

```
    {
      "chunk_id": "c06",
      "texto": "Evalué Redis vs Memcached considerando consistencia eventual...",
      "seccion": "2.2",
      "tipo_fuente": "principal",
      "peso_fuente": 1.0,
      "similitud": 0.82
    } ] }
```

Bloque de Código 7: Output conceptual Capa 5
Fuente: Elaboración Propia

Además, la Capa 5 deja disponibles los datos necesarios para calcular posteriormente la métrica R, correspondiente a la relevancia semántica de la evidencia recuperada. Para ello, conserva los puntajes de similitud coseno de los chunks seleccionados para cada competencia. Esta métrica no se calcula de forma definitiva en esta capa, ya que las citas utilizadas como respaldo serán definidas posteriormente por la Capa 6. Sin embargo, los valores de similitud entregados por Capa 5 permiten que la Capa 7 calcule R como el promedio de similitud de los chunks recuperados.

Adicional de entregar la evidencia candidata a la Capa 6, esta capa actualiza el reporte de procesamiento con información de trazabilidad por competencia. En particular, registra si la recuperación fue ejecutada para la competencia, cuántos chunks fueron recuperados, si fue necesario ampliar la búsqueda a fuentes secundarias o contextuales y si la competencia quedó sin evidencia suficiente. Esta información permite distinguir entre una competencia omitida y una competencia procesada correctamente, pero con baja o nula evidencia disponible en el informe.

```
"trazabilidad_recuperacion": {
  "recuperacion_ejecutada": true,
  "chunks_recuperados": 2,
  "expansion_fuentes": false,
  "estado_capa_5": "OK"
}
```

Bloque de código 8: Reporte de trazabilidad de la capa 5
Fuente: Elaboración propia

3.1.6 Capa 6: Evaluación mediante modelo de lenguaje

Como parte del loop iterativo por competencia, el LLM se encarga de asignar un nivel de competencia a partir de la evidencia textual recuperada en la etapa anterior. El proceso inicia cuando la capa recibe:

- los fragmentos de texto seleccionados por C5, ordenados por similitud semántica
- la descripción de la competencia a evaluar
- la rúbrica de evaluación estructurada en cuatro niveles (0: sin evidencia, 1: mención superficial, 2: uso concreto con resultados, 3: dominio técnico con reflexión e impacto).

El sistema construye un prompt dinámico que incluye estos elementos de contexto, las instrucciones explícitas de evaluación y el formato de salida esperado. A continuación se muestra un ejemplo del prompt que se le entrega al LLM.

```
ROL Y CONTEXTO
Eres un evaluador académico especializado en competencias de ingeniería de software.
Tu tarea es evaluar el desempeño de un estudiante en una competencia específica basándote
ÚNICAMENTE en la evidencia textual proporcionada. No debes hacer suposiciones ni inferencias
```

```
más allá de lo que el texto explícitamente comunica.

COMPETENCIA A EVALUAR
- Nombre: Desarrolla software y gestiona proyectos
- Descripción: El estudiante es capaz de diseñar, implementar y gestionar soluciones de
  software utilizando metodologías, herramientas y prácticas profesionales. Esto incluye la
  toma de decisiones técnicas fundamentadas, la aplicación de patrones de diseño, la
  gestión de dependencias y la evaluación de trade-offs entre alternativas tecnológicas.

RÚBRICA DE EVALUACIÓN
Debes asignar UN ÚNICO nivel según los siguientes criterios:
Nivel 0: Sin evidencia
- No hay mención de la competencia en el informe.
- No se identifica ningún trabajo relacionado con desarrollo de software o gestión de proyectos.
Nivel 1: Mención superficial o teórica
- El estudiante menciona herramientas, lenguajes o conceptos relacionados, pero sin
  demostrar aplicación práctica.
- Se describe una actividad sin profundidad técnica ni resultados tangibles.
Nivel 2: Uso concreto con resultados descritos
- El estudiante implementa soluciones de software con herramientas específicas.
- Se describen resultados tangibles (reducción de tiempo, mejora de funcionalidad, etc.).
- Hay evidencia de decisiones técnicas, pero sin reflexión profunda sobre alternativas.
Nivel 3: Dominio técnico con reflexión e impacto
- El estudiante demuestra dominio técnico profundo: evalúa alternativas, justifica decisiones,
  considera trade-offs.
- Implementa soluciones con patrones de diseño, seguridad, escalabilidad o mantenibilidad.
- Reflexiona críticamente sobre las decisiones tomadas.
- Demuestra impacto cuantificable y cualitativo en el proyecto o equipo.

EVIDENCIA RECUPERADA
A continuación se presentan los fragmentos de texto extraídos del informe del estudiante,
ordenados por relevancia semántica (similitud coseno con la competencia). Cada fragmento
está identificado y acompañado de su puntuación de similitud.
Fragmento 1 (ID: c05, Similitud: 0.89) "Implementé los endpoints con FastAPI..."
Fragmento 2 (ID: c06, Similitud: 0.82) "Evalué Redis vs Memcached..."
Fragmento 3 (ID: c04, Similitud: 0.76) "La actividad más desafiante fue ..."
Fragmento 4 (ID: c09, Similitud: 0.71) "La API redujo los tiempos de consulta en un ..."

FORMATO DE SALIDA
Debes responder ÚNICAMENTE con un objeto JSON válido, sin texto adicional ni explicaciones
fuera del JSON. El JSON debe tener exactamente esta estructura:
{
  "nivel": <entero entre 0 y 3>,
  "justificacion": "<string de máximo 2 oraciones>",
  "citas": ["<cita_1>", "<cita_2>", "<cita_3>", ...],
  "p": [<float>, <float>, <float>, <float>],
}
Notas sobre el formato:
- El campo "citas" debe ser un array de strings. Cada string debe ser un fragmento
  textual exacto extraído de los fragmentos proporcionados.
- Si no hay citas disponibles (Nivel 0), el array "citas" puede estar vacío: []
- p: La distribución de probabilidad. Donde cada elemento de p representa la probabilidad de certeza en ese
nivel. niveles=[0,1,2,3]. Debes considerar que la suma de estas probabilidades debe ser 1
- El nivel asignado debe ser coherente con p, es decir, si asignas nivel 1, entonces la probabilidad para ese
nivel debe ser el mayor (ejemplo: [0.1,0.8,0,0.1] )
```

Bloque de código 9: Prompt al LLM
Fuente: Elaboración propia

El LLM procesa el prompt y genera una respuesta estructurada que contiene cuatro elementos fundamentales: el **nivel asignado**, un valor entero entre 0 y 3, una **justificación textual** que explica brevemente por qué se asignó

ese nivel, un **conjunto de citas textuales** exactas extraídas de los fragmentos recuperados, y una **distribución de probabilidad** asociada a los cuatro niveles posibles.

Como output, la Capa 6 entrega una evaluación estructurada para la competencia procesada en esa iteración del loop. Esta evaluación corresponde a un objeto JSON que contiene el nivel asignado, la justificación, las citas textuales utilizadas como respaldo y la distribución de probabilidad p.

La salida de C6 se verifica estructuralmente para asegurar que pueda ser utilizada por las capas posteriores. Esta verificación comprueba que el JSON tenga el formato esperado, que el nivel esté dentro del rango permitido entre 0 y 3, que las probabilidades sumen 1 y que las citas correspondan a fragmentos presentes en la evidencia recuperada. Esta revisión es técnica y no corresponde a la validación final del evaluador humano.

Una vez realizada esta verificación, la evaluación no se envía de forma aislada a la Capa 7. En cambio, el orquestador del pipeline mantiene un registro consolidado de resultados donde almacena, para cada competencia procesada, tanto la evaluación generada por el LLM como la evidencia que la respalda. Este registro acumula iterativamente los datos de cada competencia conforme el loop avanza: para cada competencia se conserva la decisión del LLM (nivel, justificación, citas, distribución de probabilidad) junto con los metadatos de la evidencia recuperada (identificadores de chunks, secciones de origen, tipo de fuente, pesos y puntajes de similitud coseno).

A medida que el loop iterativo procesa cada competencia de la matriz reducida, el orquestador va agregando una nueva entrada al registro consolidado. De esta forma, el registro crece de manera incremental, acumulando la información de todas las competencias evaluadas sin necesidad de generar archivos intermedios separados.

Una vez que todas las competencias han sido evaluadas y el loop ha completado su ejecución, el registro consolidado contiene un conjunto completo de resultados: una entrada por cada competencia procesada, con su evaluación y su evidencia asociada. Este conjunto completo se entrega como input a la Capa 7, donde se consolidan los resultados finales y se calculan las métricas de confianza, pertinencia y sustento correspondientes

Además, esta capa actualiza el reporte de procesamiento con información de trazabilidad asociada a la evaluación de la competencia. En particular, registra si el LLM generó un dictamen estructurado, si la salida cumplió con el formato esperado y si las citas entregadas corresponden a fragmentos presentes en la evidencia recuperada. Esta información permite verificar posteriormente que la competencia completó el flujo mínimo de evaluación y puede ser considerada dentro del cálculo de cobertura del pipeline.

```
{
  "competencia_id": "C13",
  "estado_capa_6": "OK",
  "dictamen_generado": true,
  "formato_llm_ok": true,
  "citas_validas": true
}
```

Bloque de código 10: Reporte de trazabilidad de la capa 6
Fuente: Elaboración propia

3.1.7 Capa 7: Agregación y generación de resultados

En la última capa del pipeline, el sistema recibe como entrada los resultados acumulados generados durante el loop por competencia. Esta estructura contiene un registro por cada competencia evaluada, donde se almacena tanto la evidencia recuperada en la Capa 5 como la evaluación estructurada generada en la Capa 6. Por lo tanto, la Capa 7

no recibe únicamente el resultado del LLM, sino el conjunto completo de datos de todas las competencias a evaluar.

La función de esta capa no es volver a evaluar el informe ni recuperar nueva evidencia, sino agregar, ordenar y transformar los resultados obtenidos en una estructura interna organizada. A partir de esta información, la Capa 7 prepara dos niveles de salida: una vista preliminar orientada al evaluador y un reporte de procesamiento orientado a trazabilidad, auditoría y cálculo posterior de KPI.

La **vista preliminar** contiene la información necesaria para que el evaluador revise la evaluación de manera clara y sin sobrecarga técnica. Para cada competencia, se presenta el nivel asignado, las citas utilizadas como respaldo, la justificación generada, las secciones fuente de la evidencia y un estado general de revisión, por ejemplo “respaldo suficiente” o “requiere revisión”. Esta vista busca apoyar la toma de decisión del evaluador, mostrando los elementos interpretables de la evaluación sin exponer necesariamente todas las métricas internas del sistema.

En paralelo, la Capa 7 calcula y conserva **métricas internas** que permiten caracterizar el grado de sustento de cada evaluación. En particular, calcula la confianza del modelo a partir de la distribución de probabilidad entregada por la Capa 6. Esta confianza se obtiene mediante entropía: si el modelo concentra la mayor parte de la probabilidad en un solo nivel, la confianza es mayor, pero si reparte la probabilidad entre varios niveles posibles, la confianza disminuye. Por lo tanto, esta métrica no indica que la evaluación sea necesariamente correcta, sino qué tan definido fue el dictamen del modelo.

A partir de los datos disponibles en los resultados acumulados, la Capa 7 también calcula los componentes necesarios para el JPC de cada competencia. Este índice corresponde a una medida de sustento de la evaluación generada por el sistema, considerando la cantidad de citas utilizadas (C), la pertinencia de las secciones desde donde proviene la evidencia (S), la similitud semántica de los fragmentos recuperados (R) y la confianza del modelo (F). Es importante destacar que este JPC se calcula por competencia y no corresponde todavía al KPI agregado definido previamente. Además, el JPC se interpreta como una métrica de sustento únicamente para competencias aprobadas o validadas, ya que su propósito es medir el respaldo de una competencia considerada demostrada. En competencias rechazadas o no aprobadas, el JPC no aplica para el cálculo agregado. El KPI final puede calcularse posteriormente usando los JPC individuales de las competencias aprobadas.

En esta etapa, la Capa 7 **actualiza el reporte de procesamiento** incorporando las métricas internas calculadas para cada competencia. En particular, agrega los componentes C, S, R y F, el JPC individual, el estado de cobertura de la competencia y los estados de procesamiento acumulados durante las capas anteriores. Esta información no se muestra necesariamente en la vista preliminar del evaluador, pero queda disponible como trazabilidad técnica para auditar el funcionamiento del sistema y calcular posteriormente los KPI del MVP.

El output de esta capa corresponde a la estructura interna ordenada, lista para ser utilizada por la vista preliminar de la aplicación o por una etapa posterior de exportación.

Por lo tanto, la Capa 7 funciona como una etapa de consolidación y preparación de datos: recibe la colección completa de resultados por competencia, calcula las métricas pertinentes, separa la información entre vista de revisión y reporte técnico, y deja disponible todo lo necesario para la revisión del usuario y la posterior generación del archivo final.

```
{
  "vista_preliminar": {
    "resultados_competencias": [
      {
        "competencia_id": "C13",
```

```
        "competencia_nombre": "Desarrolla software",
        "nivel": 3,
        "nivel_label": "Destacado",
        "secciones_fuente": ["2.2", "2.2", "2.2", "3.2"],
        "citas": [
          "Evalué Redis vs Memcached considerando consistencia eventual",
          "Error rate bajó de 12% a 0.3%",
          "Implementé autenticación JWT, rate limiting y manejo de errores"
        ],
        "justificacion": "Estudiante demuestra dominio técnico concreto y toma decisiones fundamentadas.",
        "estado_revision": "respaldo_suficiente"
      },
      {
       ...
      }
    ]
  },
  "reporte_procesamiento": {
    "est_id": "EST-0042",
    "tipo_practica": "Professional",
    "secciones_detectadas": ["abstract", "2.1", "2.2", "3.1", "3.2", "4"],
    "secciones_ausentes": [],
    "trazabilidad_competencias": [
      {
        "competencia_id": "C13",
        "embedding_generado": true,
        "similitud_calculada": true,
        "recuperacion_ejecutada": true,
        "chunks_recuperados": 4,
        "dictamen_generado": true,
        "estado_cobertura": "procesada",
        "p0": 0.01,
        "p1": 0.04,
        "p2": 0.10,
        "p3": 0.85,
        "C_cobertura_citas": 1.00,
        "S_pertinencia_seccion": 1.00,
        "R_similitud_promedio": 0.80,
        "F_confianza": 0.70,
        "JPC": 0.875
      }
    ],
    "tiempos": {
      "T_procesamiento_automatico_min": null,
      "T_revision_humana_min": null,
      "T_ajustes_min": null,
      "T_IA_total_min": null
    }
  }
}
```


Bloque de código 11: Output conceptual de la capa 7
Fuente: Elaboración propia

Una vez finalizada la ejecución de la Capa 7, el orquestador de la aplicación actualiza el reporte de procesamiento con el tiempo asociado a la primera ejecución automática del pipeline, es decir, el tiempo transcurrido desde la carga del informe hasta la generación de la primera vista preliminar. Este valor se registra como *T_procesamiento_automatico_min* y permanece fijo, incluso si posteriormente el usuario solicita ajustes que reactiven parcial o totalmente algunas capas del sistema. Después de esta actualización, el campo de tiempos queda de la siguiente forma:

```
"tiempos": {
  "T_procesamiento_automatico_min": 12,
  "T_revision_humana_min": null,
  "T_ajustes_min": null,
  "T_IA_total_min": null }
```

Bloque de código 12: campo de tiempo actualizado post capa 7
Fuente: Elaboración propia

3.1.8 Human in the Loop: Revisión.

El mecanismo de Human-in-the-Loop (HITL) permite al evaluador revisar, validar y modificar las evaluaciones generadas automáticamente hasta alcanzar un estado satisfactorio. Este proceso transforma la evaluación de un acto binario (aceptar o rechazar el resultado completo) en un diálogo iterativo donde el sistema y el evaluador colaboran para refinar y mejorar continuamente las decisiones.

El proceso inicia cuando se presentan los resultados derivados de la capa 7. El evaluador revisa competencia por competencia, evaluando si la clasificación, justificación y evidencia son adecuadas. En esta etapa, el rol del usuario es fundamental, ya que para cada competencia, el evaluador tiene tres opciones: aceptar el resultado tal como está, solicitar modificaciones específicas, o rechazar la evaluación.

Aceptar
Cuando el evaluador selecciona aceptar, el sistema marca la competencia como validada y avanza a la siguiente sin realizar cambios. La evaluación se almacena en su estado actual con un registro de timestamp de validación.

Modificar
En caso de solicitar modificaciones (por ejemplo, "cambiar el nivel de C13 a 2" o "agregar una cita específica"). El sistema luego descompone la solicitud en instrucciones atómicas mediante un **clasificador** [Figura1], en donde se identifica la naturaleza del dato afectado (nivel, cita, sección, umbral) y lo enruta a la capa correspondiente:

- Cambios de nivel se dirigen a la Capa C6 (Evaluador LLM).
- Adiciones de evidencia/citas se enrutan a la Capa C5 (Recuperación semántica).
- Ajustes de estructura van a la Capa C2 (Parser).
- Modificaciones de métricas apuntan a la Capa C7.

El ***clasificador*** es un componente que analiza el texto natural del usuario y lo convierte en una instrucción estructurada que especifica: el tipo de modificación solicitada, los parámetros específicos (qué dato cambiar, a qué valor, etc.), y la capa destino donde debe procesarse.

El sistema mantiene un conjunto de patrones predefinidos que mapean tipos de solicitudes a capas específicas. Estos patrones permiten reconocer variaciones en la forma en que el evaluador expresa sus solicitudes. Por ejemplo, "*cambiar nivel a 2*", "*el nivel debería ser 2*", "*asignar nivel 2*" y "*modificar a nivel 2*" son todas formas diferentes de expresar la misma intención, que el sistema reconoce como una solicitud de *cambio de nivel.*

El proceso de clasificación funciona en dos fases. En la primera fase, el sistema intenta hacer coincidir la solicitud del usuario con los patrones predefinidos. Si la solicitud coincide exactamente con uno de los patrones conocidos, el sistema extrae automáticamente los parámetros relevantes (por ejemplo, el nuevo nivel, la nueva cita, el número de cita a remover, etc.). Si la solicitud no coincide con ningún patrón predefinido o es ambigua, el sistema activa una segunda fase donde utiliza un modelo de lenguaje ligero para interpretar la intención del usuario de manera más flexible.

Una vez identificado el tipo de modificación, el sistema consulta una tabla de enrutamiento que especifica a qué capa debe dirigirse cada tipo de solicitud:

Cambios de nivel se dirigen a la Capa C6 (Evaluador LLM). Cuando el evaluador solicita cambiar el nivel asignado, el sistema envía la instrucción a C6, donde se puede regenerar la justificación basada en el nuevo nivel.

Adiciones de evidencia/citas se enrutan a la Capa C5 (Recuperación semántica). Cuando el evaluador desea agregar una cita adicional que considera relevante, el sistema valida que esa cita existe en el informe original, calcula su similitud semántica con respecto a la competencia evaluada, y la integra en la lista de fragmentos recuperados.

Remociones de citas también se enrutan a la Capa C5 (Recuperación semántica). El sistema identifica cuál cita debe ser removida, la elimina de la lista de fragmentos, y propaga el cambio hacia C7 para recalcular las métricas afectadas.

Cambios de justificación se dirigen a la Capa C6 (Evaluador LLM). Cuando el evaluador reescribe la justificación, el sistema actualiza el texto sin afectar los componentes de confianza, ya que la justificación es una explicación textual que no impacta el cálculo de métricas.

Ajustes de estructura van a la Capa C2 (Parser). En casos excepcionales donde el evaluador identifica que el mapa de relevancia está incorrecto (por ejemplo, una sección fue clasificada como secundaria cuando debería ser principal), la solicitud se enruta a C2 para actualizar la estructura del análisis.

Una vez que la instrucción llega a su capa destino, se activa una conexión residual (inspirado en el funcionamiento de la ResNet), es decir, el dato modificado se inyecta directamente en ese nodo, sobrescribiendo el valor anterior. A partir de ese punto, el flujo se repropaga hacia abajo (hacia capas más profundas) para recalcular todas las dependencias derivadas. Por ejemplo, si se modifica un chunk en C5, este nuevo dato fluye automáticamente a C6 para re-evaluar el nivel y luego a C7 para recalcular el JPC y la confianza, garantizando la consistencia interna sin necesidad de reprocesar las capas anteriores (C1-C4) que no fueron afectadas. Este mecanismo asegura que la intervención humana se integre de manera estructural en la lógica del modelo, manteniendo la integridad de los cálculos mientras el evaluador mantiene el control final sobre la evaluación.

Rechazar

Cuando el evaluador selecciona rechazar, el sistema marca la competencia como no aprobada. A diferencia de las modificaciones iterativas, el rechazo es una decisión final que no requiere re-procesamiento de capas. Sin embargo, el sistema captura la razón del rechazo y permite al evaluador asignar un nuevo nivel. El evaluador puede rechazar una competencia por varias razones: la evidencia es genérica y no demuestra realmente la competencia, las citas son superficiales o fuera de contexto, el nivel asignado no refleja el desempeño real del estudiante, o simplemente el evaluador considera que la justificación no es convincente.

Cuando el evaluador inicia el proceso de rechazo, el sistema presenta una interfaz que solicita dos piezas de información. Primero, requiere una razón del rechazo, que es un campo de texto donde el evaluador explica por qué rechaza la evaluación automática. Esta razón se registra en la auditoría y es crucial para entender las limitaciones del sistema y para mejorar futuras evaluaciones. Segundo, el sistema permite al evaluador asignar un nivel final, que generalmente es inferior al nivel asignado por el sistema. El evaluador puede opcionalmente mantener el nivel asignado por el sistema pero marcar la competencia como rechazada

Registro de tiempos y actualización del reporte de procesamiento

Durante la etapa Human-in-the-Loop, el reporte de procesamiento se actualiza para reflejar siempre la versión vigente de la evaluación. Por lo tanto, si un ajuste modifica la evidencia, las citas, el nivel asignado o las métricas asociadas, los valores correspondientes en el reporte se recalculan y se reemplazan por los nuevos valores. Esto permite que el reporte final represente el estado validado de la evaluación, y no una versión intermedia generada antes de la revisión humana.

Sin embargo, el sistema no elimina la trazabilidad de los cambios realizados. Cada modificación queda registrada en un historial de ajustes, donde se almacena la solicitud original del usuario, la competencia afectada, las instrucciones interpretadas por el sistema, las capas reejecutadas y el resultado del ajuste. De esta manera, el reporte de procesamiento mantiene dos niveles de información: por una parte, el estado final vigente de cada competencia, y por otra, el historial de eventos que permite auditar cómo se llegó a ese resultado.

```
{
  "reporte_procesamiento": {
    "trazabilidad_competencias": [
      {
        "competencia_id": "C13",
        "estado_final": "modificada_y_validada",
        "chunks_recuperados": 5,
        "C_cobertura_citas": 1.00,
        "S_pertinencia_seccion": 1.00,
        "R_similitud_promedio": 0.83,
        "F_confianza": 0.72,
        "JPC": 0.89
      }
    ],
    "historial_ajustes": [
      {
        "ajuste_id": "ajuste_01",
        "competencia_id": "C13",
        "solicitud_usuario": "Considerar lo descrito en el Abstract",
        "capas_reprocesadas": ["C5", "C6", "C7"],
        "duracion_min": 5,
        "resultado": "reporte_actualizado"
      }
    ],
```

Bloque de código 13: Representación conceptual de registro de ajustes durante HITL
Fuente: Elaboración propia.

En esta etapa también se completan los tiempos necesarios para calcular el KPI de reducción del tiempo de revisión. El tiempo de procesamiento automático inicial ya fue registrado por el orquestador al finalizar la primera ejecución de la Capa 7 y permanece fijo. A partir de la revisión humana, el sistema registra el tiempo que el evaluador dedica a revisar la vista preliminar y el tiempo asociado a los ciclos de ajuste. Si el usuario solicita modificaciones, cada ciclo de ajuste se registra de forma independiente, indicando qué capas fueron reactivadas y cuánto duró el reprocesamiento.

Al finalizar la validación humana, el reporte de procesamiento contiene el tiempo de procesamiento automático inicial, el tiempo de revisión humana, el tiempo acumulado de ajustes y el tiempo total con IA. Esta separación permite distinguir entre la primera generación automática de resultados y los reprocesamientos derivados de la interacción del evaluador.

```
"tiempos": {
  "T_procesamiento_automatico_min": 12,
  "T_revision_humana_min": 35,
  "T_ajustes_min": 8,
  "T_IA_total_min": 55
}
```

Bloque de código 14: Representación conceptual de registro de tiempos
Fuente: Elaboración propia

3.1.9 Salida final: archivo .xlsx

Una vez que la Capa 7 ha consolidado todos los resultados y métricas, el sistema procede a la generación de la salida, cuya función es transformar la estructura interna ordenada en un archivo Excel (.xlsx) estructurado y legible para el usuario final. Esta capa actúa como un serializador que convierte los datos JSON internos en un formato tabular optimizado para revisión, auditoría y toma de decisiones.

El archivo de salida contiene dos hojas denominadas "Resultados de Evaluación" y "Reporte de Procesamiento", cada una con un propósito específico

Hoja 1: Resultados de evaluación

Esta hoja presenta la información de evaluación de manera clara y orientada al usuario. Cada fila corresponde a una competencia evaluada, y las columnas contienen los datos esenciales para la toma de decisión.

ID	Competencia_Nombre	Nivel	Justificación	Cita_1	Sección_Cita_1	Cita_2	Sección_Cita_2	Cita_3	Sección_Cita_3	JPC_Porcentaje	Estado_Revision	Timestamp_Validación
C13	Desarrolla software y gestior	3	El estudiante	Evalué Redis	2.2	Error rate bajó	3.2	Implementé a	2.2	87.50%	Respaldo suficiente	2026-05-03 14:32
C9	Trabajo en equipo y colabora	2	El estudiante	Coordiné con	2.1	Participé en re	3.1			72.00%	Respaldo suficiente	2026-05-03 14:35
C5	Comunicación efectiva	1	El estudiante	El equipo se c	2.1						Requiere Revision	2026-05-03 14:38
C3	Análisis y resolución de prob	3	El estudiante	Identifiqué qu	2.2	Propuse una a	3.1	Implementé p	2.2	85.00%	Respaldo suficiente	2026-05-03 14:40
C7	Aprendizaje continuo	2	El estudiante	Aprendí FastA	2.2	Investigué sot	2.1			70.50%	Respaldo suficiente	2026-05-03 14:42

Figura 4: .xlsx generado (Hoja 1)
Fuente: Elaboración propia

La información se inserta de manera secuencial, fila por fila, las citas se muestran en su forma completa. El campo Estado_Revisión indica si la evaluación fue aprobada automáticamente ("Respaldo Suficiente") o requiere intervención humana ("Requiere Revisión"). El JPC_Porcentaje se presenta en formato visual (mediante un porcentaje). Las columnas están coloreadas por defecto: verde para "Respaldo Suficiente", amarillo para "Requiere Revisión", rojo para "No Aprobada".

Hoja 2: Reporte de procesamiento

Esta hoja contiene información técnica de trazabilidad, auditoría y cálculo de KPIs. Está orientada a administradores, auditores y para el cálculo posterior de indicadores de desempeño del MVP

ANÁLISIS DE SECCIONES	
Secciones Detectadas	abstract, 2.1, 2.2, 3.1, 3.2, 4
Secciones Ausentes	(ninguna)
Total Secciones Esperadas	6
Total Secciones Encontradas	6
ESTADÍSTICAS DE COBERTURA	
Total Competencias Evaluadas	5
Competencias Aprobadas (JPC ≥ 0.60)	4
Competencias No Aprobadas (JPC < 0.60)	1
Tasa de Aprobación	80.00%
MÉTRICAS AGREGADAS	
JPC Promedio	0.751
JPC Máximo	0.875 (C13)
JPC Mínimo	0.505 (C5)
Confianza Promedio (F)	0.68
Similitud Promedio (R)	0.76
TIEMPOS (KPI 2)	
Tiempo Procesamiento Automático	12 minutos
Tiempo Revisión Humana (HITL)	35 minutos
Tiempo Ajustes	8 minutos
Tiempo Total con IA	55 minutos
Tiempo Manual Estimado (Línea Base)	180 minutos
Reducción de Tiempo (RTR)	69.40%
TRAZABILIDAD POR COMPETENCIA	
Competencia	Embedding_Generado
C13	✓
C9	✓
C5	✓
C3	✓
C7	✓

Figura 5: .xlsx generado (Hoja 2)
Fuente: Elaboración propia

3.2 Trazabilidad de Datos para KPIs

Además del flujo principal entre capas, el sistema conserva ciertos datos internos de trazabilidad. Estos datos no corresponden necesariamente a la evaluación final del estudiante, sino a información generada durante el procesamiento que permite auditar el funcionamiento del pipeline, construir el reporte de procesamiento y calcular posteriormente los KPI definidos para el MVP.

En primer lugar, para calcular el KPI de cobertura de evaluación de competencias, el sistema debe registrar si cada competencia de la matriz reducida fue efectivamente procesada. Para esto, se conserva una traza por competencia que indica si la competencia fue incluida en la lista de evaluación, si se generó su embedding, si se calcularon sus similitudes contra los chunks del informe, si se ejecutó la recuperación de evidencia y si se obtuvo un dictamen estructurado por parte del LLM. Esta información permite distinguir entre una competencia omitida por error y una competencia procesada sin evidencia suficiente. Por ejemplo, si el sistema analiza una competencia, busca evidencia y finalmente asigna nivel 0 por ausencia de respaldo textual, esa competencia sí cuenta como procesada.

En segundo lugar, para calcular el KPI de reducción del tiempo de revisión, el sistema debe registrar los tiempos asociados al uso de la herramienta. Esto incluye el tiempo de procesamiento automático del pipeline, el tiempo dedicado por el evaluador a revisar la vista preliminar, el tiempo usado en ajustes o modificaciones y el tiempo total antes de validar la salida final. Estos valores permiten comparar el tiempo total con IA frente al tiempo manual estimado definido previamente.

Finalmente, para calcular el KPI asociado al sustento de la evaluación mediante JPC, el sistema debe conservar los componentes generados durante el análisis de cada competencia aprobada o validada. Esto incluye la cantidad de citas utilizadas, las secciones de origen de la evidencia, la similitud semántica promedio de los chunks recuperados, la confianza del modelo y el JPC individual. El JPC no aplica a competencias rechazadas o no aprobadas, ya que su propósito es medir el grado de sustento de una competencia considerada demostrada, no cuantificar la ausencia de evidencia. Por esta razón, el reporte de procesamiento debe registrar el estado final de cada competencia e indicar si esta será considerada o no dentro del cálculo agregado del KPI asociado al JPC.

La siguiente tabla resume los datos necesarios para habilitar el cálculo posterior de los KPIs definidos:

KPI asociado	**Datos requeridos**	**Dónde se generan**
KPI 1: Cobertura de evaluación de competencias	Competencias esperadas, competencias procesadas, estado de embedding, estado de recuperación, dictamen generado	Capa 1, Capa 4, Capa 5 y Capa 6
KPI 2: Reducción del tiempo de revisión	Tiempo de procesamiento automático, tiempo de revisión humana (HITL), tiempo de ajustes, tiempo total con IA	Orquestador del pipeline y etapa HITL
KPI 3: Sustento mediante JPC	Citas, secciones fuente, similitudes, confianza del modelo, C, S, R, F, JPC individual, estado final de la competencia y campo de aplicabilidad del JPC	Capa 5, Capa 6, Capa 7 y etapa HITL

Tabla 1: Datos de trazabilidad requeridos para el cálculo de KPIs
Fuente: Elaboración Propia

En caso de modificaciones durante HITL, los datos usados para KPI se actualizan según el estado final validado, mientras que las versiones intermedias quedan registradas únicamente en el historial de ajustes.

4. Supuestos Críticos del MVP

El MVP busca validar una serie de supuestos críticos sobre la utilidad y factibilidad de usar informes de práctica como fuente de evidencia para evaluar competencias del perfil de egreso.

En primer lugar, se busca validar si una evaluación preliminar automatizada puede ser útil para el evaluador humano. Es decir, si presentar niveles sugeridos, citas, justificaciones y métricas de sustento ayuda a reducir la carga de revisión, mejorar la trazabilidad y facilitar la toma de decisiones curriculares.

En segundo lugar, se busca validar si el mecanismo Human-in-the-Loop aporta valor al proceso. Esto significa comprobar si el evaluador puede revisar, corregir y validar los resultados de manera clara, y si sus ajustes permiten mejorar la calidad final de la evaluación sin perder consistencia entre evidencia, nivel asignado y justificación.

Otro supuesto que se busca validar si los informes de práctica contienen evidencia suficiente para identificar competencias del perfil de egreso. Esto implica comprobar si, dentro del texto elaborado por los estudiantes, existen descripciones de actividades, decisiones, resultados o reflexiones que permitan reconocer habilidades específicas más allá de la aprobación general del informe.

También se busca validar si esta evidencia puede ser detectada y asociada correctamente a competencias mediante técnicas de análisis textual y semántico. En particular, se evaluará si el uso de embeddings, similitud coseno y

recuperación de fragmentos permite encontrar evidencia relevante incluso cuando el estudiante no utiliza exactamente las mismas palabras de la matriz de competencias.

Adicionalmente, se busca validar si una implementación local e interactiva es suficiente para probar el flujo principal del sistema. Esto implica comprobar si una herramienta acotada, sin despliegue web permanente ni integración institucional inicial, permite demostrar valor en la solución.

Finalmente, el MVP busca validar si la información generada puede transformarse en datos estructurados útiles para la gestión curricular. En concreto, se espera comprobar si los resultados exportados por competencia permiten avanzar desde una lógica de aprobación/reprobación hacia una visión más granular, trazable y comparable del cumplimiento del perfil de egreso.

5. Conclusión

El MVP propuesto permite transformar los informes de práctica, actualmente utilizados principalmente dentro de una lógica de aprobación o reprobación, en una fuente de información estructurada para analizar el cumplimiento del perfil de egreso. A través de un pipeline de análisis documental, el sistema extrae texto, organiza el informe, recupera evidencia relevante, evalúa competencias específicas y genera resultados trazables por cada eje formativo.

La propuesta se mantiene acotada a una implementación realista, planteada como una aplicación local e interactiva capaz de procesar el cumplimiento del perfil de egreso en informes individuales, generar una vista preliminar revisable y exportar un archivo final en formato .xlsx. De esta manera, el MVP busca validar el flujo principal de evaluación por competencias en un entorno controlado y abordable.

Uno de los principales aportes del sistema es incorporar una capa adicional de análisis al proceso de revisión. La evaluación preliminar generada por el modelo permite entregar niveles sugeridos, citas, justificaciones y métricas de sustento, mientras que el mecanismo Human-in-the-Loop asegura que el evaluador pueda revisar, ajustar y validar los resultados antes de considerarlos finales. Así, la solución combina automatización, trazabilidad y criterio humano dentro de un mismo flujo.

Además, el diseño propuesto es consistente con los KPI definidos para el MVP, ya que registra la cobertura del procesamiento por competencia, los tiempos asociados a la revisión y los componentes necesarios para calcular el sustento de las evaluaciones aprobadas mediante JPC. Esto permite no solo evaluar el desempeño de los estudiantes, sino también evaluar el desempeño del propio sistema y medir los objetivos de la solución.

En conjunto, el MVP constituye una solución concreta, delimitada y evaluable para avanzar desde una revisión binaria del perfil de egreso, hacia una evaluación más granular, trazable y basada en evidencia. Su implementación permitirá validar si los informes de práctica contienen evidencia suficiente, si esta puede recuperarse semánticamente de forma útil y si los resultados generados apoyan una toma de decisiones curricular más precisa.